



会员 周边 新闻 博问 闪存 赞助商 Chat2DB

代码改变世界



注册 登录

hellokitty2

随笔 - 684, 文章 - 1, 评论 - 21, 阅读 - 102万

导航

博客园

首页

新随笔

联系

订阅 

管理

< 2025年5月 >						
日	一	二	三	四	五	六
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

公告

昵称： Hello-World3

园龄： 8年2个月

粉丝： 113

关注： 7

+加关注

搜索

pinctrl(2)——驱动实现与设备树配置

一、pinctrl子系统简介

1. pin control subsystem驱动的硬件叫做pin controller，主要功能包括：

(1) pin multiplexing，pin引脚复用。

(2) pin configuration，这些配置参数包括 pull-up/down电阻的设定， tri-state设定， drive-strength的设定。

2. pin controller这个HW block需要是device tree中的一个节点。此外， device也需要在它自己的 device tree node中描述pin control的相关内容。也就是说引脚配置在pin controller的设备树节点中，设备的节点中通过其phandle去引用这些节点去使用。

3. 每个pin configuration都是pin controller的child node，描述了client device要使用到的一组pin的配置信息。具体如何定义pin configuration是和具体的pin controller相关的。在pin controller node中定义pin configuration其目的是为了let client device引用。

4. pinctrl driver需要根据实际情况，将系统中所有的pin组织成一个struct pinctrl_pin_desc类型的数组。

5. Pin groups

在SoC系统中，为了实现特定的功能，例如SPI接口、I2C接口等。因此pin controller需要支持group功能，访问、控制多个pin，这就是pin groups。相应地，pin controller

找找看

常用链接

- 我的随笔
- 我的评论
- 我的参与
- 最新评论
- 我的标签

我的标签

- time(1)
- scanf()(1)
- Gtk(1)
- gets()(1)
- fgets()(1)
- CRC校验(1)

随笔分类

- Android (75)
- AndroidTV(1)
- Android上层(14)
- Android手机相关(11)
- Binder(10)
- C++相关(38)
- C语言相关(14)
- eBPF(1)
- git操作(12)
- GStreamer(1)
- Java(24)
- Linux shell(4)
- Linux电源管理和功耗(19)
- linux命令(10)
- Linux驱动相关(84)

subsystem需要提供一些机制，来获取系统中到底有多少groups、每个groups包含哪些pins、等等。因此，pinctrl core在struct pinctrl_ops中抽象出三个回调函数，用来获取pin groups相关信息。

```
struct pinctrl_ops {
    /*获取系统中pin groups的个数，后续的操作，将以相应的索引为单位 (类似数组的下标，个数为数组的大
    int (*get_groups_count) (struct pinctrl_dev *pctldev);
    /*获取指定group (由索引selector指定) 的名称。*/
    const char *(*get_group_name) (struct pinctrl_dev *pctldev, unsigned selector);
    /*获取指定group的所有pins (由索引selector指定) ， 结果保存在pins (指针数组) 和num_pins中。
    int (*get_group_pins) (struct pinctrl_dev *pctldev, unsigned selector, const uns
    void (*pin_dbg_show) (struct pinctrl_dev *pctldev, struct seq_file *s, unsigned
    /*将device tree中的pin state信息转换为pin map*/
    int (*dt_node_to_map) (struct pinctrl_dev *pctldev, struct device_node *np_confi
    void (*dt_free_map) (struct pinctrl_dev *pctldev, struct pinctrl_map *map, unsig
};
```

6. Pin configuration

Pin configuration 的对象是pin或者pin group，SoC中的管脚有些属性可以配置，例如上拉、下拉、高阻、驱动能力等。pinctrl subsystem使用pin configuration来封装这些功能，具体体现在struct pinconf_ops数据结构中，如下：

```
struct pinconf_ops {
    #ifdef CONFIG_GENERIC_PINCONF
    bool is generic;
    #endif
    /*获取
    且 注册信息获得) 当前配置，保存在config指针中 (配置的具体含义，
    int (*pin_config_get) (struct pinctrl_dev *pctldev, unsigned pin, unsigned long
```

联合会员

更多

随笔档案

- 2025年5月(2)
- 2025年4月(6)
- 2025年3月(8)
- 2025年2月(14)
- 2025年1月(3)
- 2024年12月(2)
- 2024年11月(5)
- 2024年10月(1)
- 2024年9月(4)
- 2024年8月(11)
- 2024年7月(14)
- 2024年6月(6)
- 2024年5月(3)
- 2024年4月(5)
- 2024年3月(6)

更多

文章分类

- Linux驱动(1)

阅读排行榜

- 1. Android.bp学习笔记(31361)
- 2. Patch打补丁学习笔记(25044)
- 3. fstab文件详解(19998)
- 4. 世界各个地区WIFI 2.4G及5G信道划分表（附无线通信频率分配表）(18389)
- 5. SDIO接口(17714)

评论排行榜

- 1. Linux内核红黑树2—移植笔记(2)

```
/*设置指定pin的配置(可以同时配置多个config, 具体意义要由相应pinctrl driver)*/
int (*pin_config_set) (struct pinctrl_dev *pctldev, unsigned pin, unsigned long
/*获取指定pin group的配置项*/
int (*pin_config_group_get) (struct pinctrl_dev *pctldev, unsigned selector, uns
/*设置指定pin group的配置项*/
int (*pin_config_group_set) (struct pinctrl_dev *pctldev, unsigned selector, uns
int (*pin_config_dbg_parse_modify) (struct pinctrl_dev *pctldev, const char *arg
void (*pin_config_dbg_show) (struct pinctrl_dev *pctldev, struct seq_file *s, ur
void (*pin_config_group_dbg_show) (struct pinctrl_dev *pctldev, struct seq_file
void (*pin_config_config_dbg_show) (struct pinctrl_dev *pctldev, struct seq_file
};
```

kernel pinctrl subsystem 并不关心configuration的具体内容是什么，它只提供pin configuration get/set的通用机制，至于get到的东西，以及set的东西，到底是什么，是 pinctrl driver 自己的事情。后面结合pin map和pin state，就能更好地理解这种设计了。

7. Pin multiplexing(对象是pin或者pin group)

Pin multiplexing 的对象也是pin或者pin group，SoC中的很多管脚可以配置为不同的功能，这称作管脚的复用（pin multiplexing，简称为pinmux）。kernel pinctrl subsystem使用struct pinmux_ops来抽象pinmux有关的操作，如下：

```
struct pinmux_ops {
/*检查某个pin是否已作它用，用于管脚复用时的互斥（避免多个功能同时使用某个pin而不知道，导致奇怪
int (*request) (struct pinctrl_dev *pctldev, unsigned offset);
/*request的反操作。*/
int (*free) (struct pinctrl_dev *pctldev, unsigned offset);
/*获取：
int (*get_function_count) (struct pinctrl_dev *pctldev);
```

- 2. 内存管理-40-watermark内存水位(1)
- 3. Android对重启原因的记录(1)
- 4. MTK Powerhal 学习笔记(1)
- 5. Linux内核机制—mutex(1)

推荐排行榜

- 1. Patch打补丁学习笔记(4)
- 2. dm-verity简介 —— (1) (3)
- 3. Android音频(9)—— 音量调节(2)
- 4. 世界各个地区WIFI 2.4G及5G信道划分表（附无线通信频率分配表）(2)
- 5. fstab文件详解(2)

最新评论

1. Re:内存管理-40-watermark内存水位
博主您好，请问文中提及到的《内存管理-39-lowmem_reserve低端预留内存》有文章链接吗

--凛冬将至*

2. Re:世界各个地区WIFI 2.4G及5G信道划分表（附无线通信频率分配表）
图片都失效了

--orz_2333

3. Re:Android对重启原因的记录
你好请问文中说的 prop.txt 在什么位置

--CCint3

4. Re:Linux regulator系统 (1)
常供电的设备树节点该怎么写呢？

--潇汀

5. Re:iowait学习笔记
在实际上IO服务时间非常长，但由于系统中同时存在CPU密集型任务掩盖了iowait这个通过什么能看出来？？

```
/*获取指定function的名称。*/
const char *(*get_function_name) (struct pinctrl_dev *pctldev, unsigned selector
/*获取指定function所占用的pin group（可以有多个）。*/
int (*get_function_groups) (struct pinctrl_dev *pctldev, unsigned selector, const
/*将指定的pin group（group_selector）设置为指定的function（func_selector）。*/
int (*set_mux) (struct pinctrl_dev *pctldev, unsigned func_selector, unsigned gr
/*下面是gpio有关的操作*/
int (*gpio_request_enable) (struct pinctrl_dev *pctldev, struct pinctrl_gpio_rar
void (*gpio_disable_free) (struct pinctrl_dev *pctldev, struct pinctrl_gpio_ran
int (*gpio_set_direction) (struct pinctrl_dev *pctldev, struct pinctrl_gpio_ran
bool strict;

};
```



注释：

(1)为了管理SoC的管脚复用，pinctrl subsystem抽象出function的概念，用来表示I2C0、UART5等功能。pin（或者pin group）所对应的function一经确定，它们的管脚复用状态也就确定了。

(2)和pin group类似，pinctrl core不关心function的具体形态，只要求pinctrl driver将SoC的所有可能的function枚举出来（格式自行定义，不过需要有编号、名称等内容），并注册给pinctrl core。后续pinctrl core将会通过function的索引，访问、控制相应的function。

(3)同一个function（如I2C0），可能可以map到不同的pin（或者pin group）上。

8. pinctrl subsystem的控制逻辑

8.1 pin state

pin(pin group)以及相应的function和configuration的组合，可以确定一个设备的一个“状态”。这个状态在pinctrl subsystem中

联合会员

8.2 pin map

`--console``xinhun`

(1) 在pinctrl subsystem中，pin state有关的信息是通过pin map收集，相关的数据结构如下：



```
/* include/linux/pinctrl/machine.h */

struct pinctrl_map {
    const char *dev_name; //device的名称。
    const char *name; //pin state的名称。
    enum pinctrl_map_type type;
    const char *ctrl_dev_name; //pin controller device的名字。
    /*
     * data, 该map需要用到的数据项， 如果map的类型是 PIN_MAP_TYPE_CONFIGS_GROUP,
     * 则为struct pinctrl_map_mux 类型的变量；
     * 如果map的类型是 PIN_MAP_TYPE_CONFIGS_PIN 或者 PIN_MAP_TYPE_CONFIGS_GROUP,
     * 则为struct pinctrl_map_configs 类型的变量。
     */
    union {
        struct pinctrl_map_mux mux;
        struct pinctrl_map_configs configs;
    } data;
};

/*
 * type取值包括:
 * PIN_MAP_TYPE_MUX_GROUP 配置管脚复用
 * PIN_MAP_TYPE_CONFIGS_PIN 配置pin
 * PIN_MAP_TYPE_CONFIGS_GROUP 配置pin group
 * PIN_MAP_TYPE_DUMMY_STATE 不需要任何配置，仅仅为了表示state的存在。
 */
```



联合会员

struct pinctrl_map_mux的定义如下：

```
struct pinctrl_map_mux {  
    const char *group; //group的名字, 指明该map所涉及的pin group。  
    const char *function; //function的名字, 表示该map需要将group配置为哪种function。  
};
```

struct pinctrl_map_configs的定义如下:

```
struct pinctrl_map_configs {  
    const char *group_or_pin; //pin或者pin group的名字。  
    unsigned long *configs; //configuration数组, 指明要将该group或pin配置成“什么样子”。  
    unsigned num_configs; //配置项的个数。  
};
```

注: 讲到这里, 应该理解为什么struct pinconf_ops中的api, 都不知道configuration到底是什么东西了吧, 因为都是pinctrl driver自己安排好的, 自产自销, 外人 (pinctrl subsystem以及consumers) 没必要理解!

(2) 通过dts生成pin map

在旧时代, kernel的bsp工程师需要在machine有关的代码中, 静态的定义pin map数组, 这一个非常繁琐且不易维护的过程。不过当kernel引入device tree之后, 事情就简单了很多: pinctrl driver确定了pin map各个字段的格式之后, 就可以在dts文件中维护pin state以及相应的mapping table。pinctrl core在初始化的时候, 会读取并解析dts, 并生成pin map。而各个consumer, 可以在自己的dts node中, 直接引用pinctrl driver定义的pin state, 并在设备驱动的相应的位置, 调用pinctrl subsystem提供的API, active或者deactive这些state。至于dts中pin map描述的格式是什么, 则完全由pinctrl driver自己决定, 因为, 最终的解析工作 (dts to map) 也是它自己做的。

9.Consumer驱动对pinctrl子系统的使用

(1) 设备树中配置

(2) 驱动中调用

联合会员

pin_get 接口, 获得一个struct pinctrl类型的handle指针, 此函

数调用如下：

```
pinctrl_get
create_pinctrl //drivers/pinctrl/core.c
pinctrl_dt_to_map //drivers/pinctrl/devicetree.c
dt_to_map_one_config
pctlops->dt_node_to_map
```

(3) 调用 pinctrl_select_state() 选择一个state，使自己的某个pin state生效。

(4) 调用pinctrl subsystem提供的其它API，pinctrl subsystem进而调用pinctrl driver提供的各种回调函数，配置pin controller的硬件。

注：在设备和驱动在探测(probe)之前，就已经调用了pinctrl_bind_pins()来获取4种state的引脚配置了，保存在struct device结构的struct dev_pin_info pins成员中，也可以从中直接获取。



```
struct dev_pin_info {
    struct pinctrl *p;
    struct pinctrl_state *default_state;
    struct pinctrl_state *init_state;
#ifdef CONFIG_PM
    struct pinctrl_state *sleep_state;
    struct pinctrl_state *idle_state;
#endif
};

really_probe
pinctrl_bind_pins
dev->pins->default_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_DEFAULT);
dev->pins->init_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_INIT);
dev->pins->sleep_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_SLEEP);
```

联合会员

```
dev->pins->idle_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_IDI
```

二、pinctrl设备树配置

(1) client device的DTS

一个典型的device tree中的外设node定义如下：

```
device-node-name {
    .....
    pinctrl-names = "sleep", "active";
    pinctrl-0 = <pin-config-0-a>; //pinctrl-x是一个句柄（phandle）列表，每个句柄指向一个pi
    pinctrl-1 = <pin-config-1-a>;
    .....
};
```

如果设备树中没有通过 pinctrl-names 字段指定state的话，由解析函数 pinctrl_dt_to_map() 可知，statename 降为pinctrl-X的X字符。

(2) pinctrl host的设备树配置

```
pinctrl@f120000 {
    .....
    uart2_ 联合会员
    ua.
    phandle = <0x261>;
```



```
        mux {
            pins = "gpio117", "gpio118";
            function = "qup2";
        };

        config {
            pins = "gpio117", "gpio118";
            drive-strength = <0x2>;
            bias-disable;
        };
    };

    uart2_sleep {
        phandle = <0x262>;
        mux {
            pins = "gpio117", "gpio118";
            function = "gpio";
        };
        config {
            pins = "gpio117", "gpio118";
            drive-strength = <0x2>;
            bias-pull-down;
        };
    };
};

};
```



举例：



联合会员

/* 1.1 设备端可以访问设备 /

```
i2c@990000 {
    compatible = "qcom,i2c-geni";
    .....
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <0x27d>;    //default状态对应下面的phandle = <0x27d>;
    pinctrl-1 = <0x27e>;    //sleep状态对应下面的phandle = <0x27e>;
    .....
};

/* 1.2 pinctrl host端的设备树配置 */
qup7_i2c_pins {
    phandle = <0x480>;

    qup7_i2c_active {
        phandle = <0x27d>;
        mux {
            pins = "gpio20", "gpio21";
            function = "qup7";
        };
        config {
            pins = "gpio20", "gpio21";
            drive-strength = <0x2>;
            bias-disable;
        };
    };

    qup7_i2c_sleep {
        phandle = <0x27e>;
        mux {
            pins = "gpio20", "gpio21";
            function = "sleep";
        };
    };
};
```

联合会员

```
        config {
            pins = "gpio20", "gpio21";
            drive-strength = <0x2>;
            bias-no-pull;
        };
    };
};
```



注：phandle是dtb反编译成dts文件后额外生成的，源设备树文件中没有这个域，对应关系如下：



1. 设备树种的配置

```
tsif1_signals_active: tsif1_signals_active {
    tsif2_clk {
        pins = "gpio73";
        function = "tsif1_clk";
    };
};

tsif1_sync_active: tsif1_sync_active {
    tsif2_sync {
        pins = "gpio76";
        function = "tsif1_sync";
        drive_strength = <2>;      /* 2 mA */
        bias-pull-down;           /* pull down */
    };
};
```

联合会员

```
tspp: msm_tspp00000000 \
```

```
compatible = "qcom,msm_tspp";
...
pinctrl-4 = <&tsif1_signals_active &tsif1_sync_active>;      /* tsif1-mode2 */
...
}
```

2.dtb反解析后的配置:

```
tsif1_signals_active {
    phandle = <0x9f>; //反编译后生成的，原始设备树种没有。

    tsif2_clk {
        pins = "gpio73";
        function = "tsif1_clk";
    };
};

tsif1_sync_active {
    phandle = <0xa0>; //dtb反编译成dts才生成的，只是这个是增加的，其它的没有变。

    tsif2_sync {
        pins = "gpio76";
        function = "tsif1_sync";
        drive_strength = <0x2>;
        bias-pull-down;
    };
};

msm_tspp@8880000 {
    compat
    ...
    pinctrl-4 = <0x9f 0xa0>; //反编译后的使用的是phandle值来代替"&节点名称"
```

联合会员

```
...
}
```



三、gpio子系统与pinctrl子系统之间的耦合关系

1. gpio subsystem是pinctrl subsystem的client，基于pinctrl subsystem提供的功能，处理GPIO有关的逻辑。

2. pinctrl中和gpio有关的后门

```
static inline int pinctrl_request_gpio(unsigned gpio) ;
static inline void pinctrl_free_gpio(unsigned gpio) ;
static inline int pinctrl_gpio_direction_input(unsigned gpio);
static inline int pinctrl_gpio_direction_output(unsigned gpio);
```

当 gpio driver 需要使用某个管脚的时候，直接调用 pinctrl_request_gpio，向pinctrl subsystem申请。pinctrl subsystem会维护一个gpio number到pin number的map，将gpio subsystem传来的gpio number转换为pin number之后，调用 struct pinmux_ops 中有关的回调函数即可：

联合会员



```
struct pinmux_ops {  
    ...  
    int (*gpio_request_enable) (struct pinctrl_dev *pctldev, struct pinctrl_gpio_range *range);  
    void (*gpio_disable_free) (struct pinctrl_dev *pctldev, struct pinctrl_gpio_range *range);  
    int (*gpio_set_direction) (struct pinctrl_dev *pctldev, struct pinctrl_gpio_range *range, unsigned int direction);  
};
```



对gpio driver来说，要做的事情就是提供gpio number到pin number的map。而pinctrl subsystem呢，做自己分内的事情即可：管理系统的pin资源，并根据gpio subsystem的请求，将某些pin设置为GPIO功能。

4. gpio number通过struct pinctrl_gpio_range结构映射到pin number

(1) gpio number和pin number的map是通过一个名称为gpio range的数据结构(pinctrl subsystem提供的)，如下：



```
struct pinctrl_gpio_range {  
    struct list_head node;  
    const char *name;  
    unsigned int id;  
    unsigned int base;  
    unsigned int pin_base;  
    unsigned const *pins;  
    unsigned int npins;  
    struct gpio_chip *gc;  
};
```



联合会员

就是：gpio controller(gc)中的gpio(base)到gpio(base + npins - 1) 和 pin controller中的pin(pin_base) 到 pin(pin_base + npins - 1) 是一一对应的。

(2) gpio range dts node的解析过程大致如下



```
devm_gpiochip_add_data(drivers/gpio/gpiolib.c)
  gpiochip_add_data
    of_gpiochip_add(drivers/gpio/gpiolib-of.c)
      of_gpiochip_add_pin_range
        gpiochip_add_pin_range(drivers/gpio/gpiolib.c)
          pinctrl_find_and_add_gpio_range(drivers/pinctrl/core.c)
            pinctrl_add_gpio_range
```



以上过程的结果，就是在相应的pin controller device的数据结构中生成了一个gpio range的链表，如下：



```
struct pinctrl_dev {
    struct list_head node;
    struct pinctrl_desc *desc;
    struct radix_tree_root pin_desc_tree;
    struct list_head gpio_ranges;
    ...
};
```



(3) gpio range的 联合会员

当gpio driver需要使用某一个gpio的时候，可以在gpiochip的request函数中，调用pinctrl core提供的

pinctrl_request_gpio()接口（参数是gpio编号），然后pinctrl core会查寻gpio ranges链表，将gpio编号转换成pin编号，然后调用pinctrl的相应接口（参数是pin编号），申请该pin的使用。这就是gpio subsystem和pinctrl subsystem的耦合。

分类: [Linux驱动相关](#)

[好文要顶](#)[关注我](#)[收藏该文](#)[微信分享](#)



Hello-World3

粉丝 - 113 关注 - 7

0

0

[+加关注](#)

[升级成为会员](#)

« 上一篇: [CPU知识学习汇总](#)
» 下一篇: [【文件系统】实现一个小的文件系统](#)

posted on 2020-05-24 18:07 Hello-World3 阅读(5719) 评论(0) 收藏 举报

[刷新页面](#) [返回顶部](#)

- 登录后才能查看或发表评论，立即 [登录](#) 或者 [逛逛](#) 博客园首页
- 【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
- 【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
- 【推荐】博客园2025新款「AI繁忙」系列T恤上架，前往周边小店选购
- 【推荐】凌霞软件回馈社区，携手博客园推出1Panel与Halo联合会员
- 【推荐】轻量又高 联合会员 加持，快人一步